A Bit-Exact, Processor-Free FPGA Adaptive Noise Canceller: Fair Comparison of LMS, NLMS and Divider-Free NLMS-Shift on Cyclone IV E

Satvik Garg[1], Aditya Gupta[1], Jaffino G[1*]

[1*]School of Electronics Engineering (SENSE), Vellore Institute of Technology, Vellore, India.

*Corresponding author(s). E-mail(s): [TBD: e-mail; author order to be confirmed by the authors];

Abstract

Adaptive noise cancellation with a reference microphone is a classical application of the least-mean-square (LMS) family, and a small field-programmable gate array (FPGA) can perform it at audio rates without any processor. This paper presents ULLAP, a 12-bit, 32-tap noise canceller for the DE2-115 board (Cyclone IV E) and its WM8731 codec. One serial core with a single shared multiplier implements LMS, normalized LMS (NLMS) with a restoring divider, and NLMS-Shift, in which a leading-one detector and a shift take the place of the division. With step sizes tuned on held-out seeds, NLMS-Shift matches NLMS within the 95% confidence interval on stationary and amplitude-step noise (22.37 and 24.33 dB over 20 seeds), outperforms LMS by about 2 dB on amplitude steps, and needs 72 rather than 130 busy cycles per sample. NLMS-Shift is also shown to be equivalent to NLMS with a level-dependent step between 0.5 and 1 times the nominal step. On real recorded noise and speech, the filters remove 11.7–14.9 dB and raise short-time objective intelligibility from 0.69–0.78 to 0.95–0.98; without speech they remove 18.5–31.9 dB, so the speech, not the noise, limits performance. A 12-bit word keeps the quantization loss below 0.4 dB on broadband noise. On the board, all 724,459 captured output samples of the three algorithms equal those of the integer model, and the measured sample rate, latency and busy cycles agree with the register-transfer-level design.

Keywords: adaptive noise cancellation, NLMS, divider-free normalisation, FPGA, fixed-point arithmetic, bit-exact verification

1 Introduction

Few places where people live or work are free of noise. An office air conditioner hums, a compressor runs beside a production line, road noise fills a car cabin, and a hearing-aid user is surrounded by the chatter of other people. If a second microphone can be placed near the noise source, the noise reaching the main microphone can be predicted from this reference signal and subtracted. The predictor is an adaptive filter: it learns the acoustic path between the two microphones while the system runs and follows the path when it changes. For this filter, the least-mean-square (LMS) algorithm and its normalized version (NLMS) remain the usual choice. Both cost only a few multiply–accumulate operations per tap and per sample, and their behavior has been studied for decades [1].

A small embedded device has to run this filter in real time, with a short and fixed delay and on very little energy. A field-programmable gate array (FPGA) suits such a task well. The datapath can be shaped around the algorithm, the latency is known to the clock cycle, and no processor or operating system is needed for scheduling. In hardware, however, LMS and NLMS are not equally easy to build. LMS uses only multipliers and adders, but its convergence speed depends on the input level. NLMS removes this dependence by dividing the step size by the input power, at the price of one division per sample. Dividers are slow and large, so recent designs approximate the division instead, with an approximate Newton–Raphson iteration [2], with canonical-signed-digit reciprocals [3], or with a binary reciprocal built from a leading-one detector and a shifter [4]. The crudest option is to round the input power to a power of two, after which the division is a plain shift. This variant is called NLMS-Shift in the rest of the paper.

Two issues in the published work motivated this study. The first concerns tuning. Competing variants are seldom tuned with equal care, so a reported gain can come from a weak baseline rather than from the algorithm. An early version of the present work made exactly this mistake: NLMS-Shift looked 5.7 dB better than NLMS until NLMS was given a sensible regularization constant, at which point the gap vanished. The second issue is the link between model and silicon. Hardware papers usually report area, clock rate and power. Such figures show that a design fits on the chip; they do not show that the chip produces the same samples as the model behind the accuracy results.

This paper presents ULLAP, a 12-bit, 32-tap adaptive noise canceller for the low-cost Terasic DE2-115 board (Intel Cyclone IV E) and its WM8731 audio codec, and uses it to address both issues. The main contributions are listed below.

1. **One serial core for three algorithms:** LMS, NLMS (with a restoring divider) and NLMS-Shift run on the same serial core and share one multiplier, so the datapath, word length and filter length are identical for all three. The core sits inside a complete audio system with no processor, closes timing at 50 MHz on Cyclone IV E, and is released together with all scripts and data.
2. **A fair comparison:** The step size and regularizer of every algorithm are chosen on held-out seeds by one common rule, and each result carries a 20-seed 95% confidence

interval. Synthetic scenarios, real recorded noise and speech, and a reproducible word-length study are all covered.

3. **A bit-exact verification chain:** The chain runs from a Python integer model through the RTL to the board wrapper (86 regression tests, codec, freeze and reset behavior included). Captures of real codec samples on the board show every output sample of all three algorithms to be identical to the model output.
4. **Two explanations backed by measurement:** The first concerns the shift. NLMS-Shift turns out to be NLMS with a step that moves between 0.5 and 1 times the nominal value as the input level changes. Running NLMS with the average of that step reproduces NLMS-Shift within 0.3 dB on broadband noise; on tonal input, however, the step jumps each time the window power crosses a power of two. The second concerns the real recordings. Once the speech is taken out, the same filters remove 18.5–31.9 dB of the same real noise, which means that the lower noise reduction with speech comes from the speech disturbing the adaptation, not from the noise.

The shift normalizer is not new, and no claim is made to it; Section 2 places it among earlier division-free designs.

The remainder of the paper is organized as follows. Section 2 reviews the problem and related work, and Section 3 outlines the system. The algorithms and the fixed-point model are described in Section 4 and the hardware in Section 5. Sections 6 and 7 cover verification and the experimental set-up, Section 8 reports the results, and Sections 9 and 10 close the paper with a discussion and conclusions.

2 Literature review

2.1 The problem: noise in daily life

Noise is now recognized as an environmental health risk, not just a nuisance. More than 113 million people in Europe live with transport noise above 55 dB(A), and traffic noise alone is estimated to cost about 1.3 million healthy life-years each year through its effects on the heart and blood vessels [5]. Hahad et al. [6] reviewed the evidence linking long-term noise exposure to mental-health disorders and discussed the biological pathways that may be responsible. Hearing loss is the other half of the picture. The World Health Organization puts the number of people living with some degree of hearing loss at about 1.5 billion and expects it to reach 2.5 billion by 2050 [7]. For these listeners, background noise is the main reason speech stays hard to follow, even with a hearing aid. Reducing noise at the microphone, or close to the ear, is useful beside roads, where urban traffic noise lies mostly below 2 kHz [8]; inside vehicle cabins [9, 10]; in ventilation ducts and air conditioners [11]; and in hearing aids [12–14]. Most of these devices are small, cheap and run on batteries, so the processing must work in real time at low cost and low energy.

2.2 Applications of adaptive noise control

Yang et al. [8] recorded road traffic noise at five sites in Beijing and built a feedforward LMS prototype that removed close to 30 dB(A) of single- and dual-frequency noise

below 2 kHz. For a passenger car, Jiang et al. [9] made the FxLMS step size depend on the reference norm and tuned the two parameters of this rule by particle-swarm optimization. Their system, running on a dSPACE board, reached up to 12 dB below 100 Hz and about 3 dB over 20–400 Hz in road tests, against −0.8 to 0.3 dB for plain FxLMS. Zhu et al. [10] drove the car's own loudspeakers to control the first four engine orders; the average reduction at 52 ear-level microphones was 6.2 dB, and 93% of the listeners in a jury test felt more comfortable with the system on. In hearing aids, Yin and Chen [12] weighted the error of a variable-step adaptive feedback canceller, which on a real device converged 35% faster than a PEM-IMLMS baseline and 60% faster than PEM-NLMS. Husstedt et al. [13] measured six commercial behind-the-ear devices and found that impulse noise reduction cuts C-weighted peak levels by up to 20 dB, about 9 dB on average. A recent systematic review of speech enhancement for hearing aids [14] concluded that deep networks do best in complex acoustic scenes, while hybrids that keep an adaptive filter strike a better balance between latency and computation on wearable devices.

2.3 Adaptive algorithms

Lu et al. surveyed a decade of active noise control algorithms in two parts, one on linear systems [1] and one on nonlinear systems [15]. In both, LMS-type updates are still the base on which most newer methods are built. One popular idea is to run a fast filter and an accurate filter side by side and combine them. Félix et al. [16] proposed a modified convex combination of FXLMS filters that converged faster and attenuated more than both FXLMS and the earlier convex scheme. Zhu et al. [11] joined an adaptive FxLMS filter and a CNN–GRU network through a convex weight to cancel harmonic noise in a bent air-conditioner pipe. A different difficulty arises when the loudspeaker leaks back into the reference microphone and the loop turns unstable; Li et al. [17] handled this case with a Youla-parameterized feedforward controller and an adaptive IIR update, tested on a duct. Each of these methods cancels better, but each adds operations, a network or a whole extra filter to every sample, which runs against what a small FPGA design needs.

2.4 Hardware realizations

Thannoon and Hashim [18] combined a fast retimed RLS filter with a slow LMS filter on an FPGA to remove power-line interference from ECG records of the MIT-BIH database, and reported 24.2% higher SNR and 50% faster convergence than conventional structures. Jagadeeswari et al. [19] applied cut-set retiming to delayed NLMS filters for audio on a Spartan-6 and shortened the critical path from 29.96 to 13.39 ns. For underwater sonar, Karakaya and Aytar [20] built semi-parallel 16-bit NLMS and variable-step LMS filters on an Artix-7; the SNR rose from 8.85 dB to 29.36 and 28.57 dB, respectively, while using 2.51% and 2.91% of the LUTs. Park and Park [21] took another route, letting a small CNN classify the noise and pick one of several fixed filters, which in simulation gave 2.3 dB more SNR than LMS with 53% less dynamic power. Abdi et al. [22] designed a narrowband controller around a variable-step LMS

notch filter on an FPGA to suppress microphonics in a superconducting accelerator cavity. Other groups worked on the arithmetic. Nagabushanam et al. [23] built a multiplier-less LMS filter from distributed arithmetic with offset-binary coding, and da Rosa et al. [24] replaced the multipliers of an LMS power-line canceller with adders and shifters, saving up to 4.98 times in power and 30.46% in area.

2.5 Removing the division from NLMS

Closest to the present study are the papers that attack the NLMS division itself. Guidotti et al. [2] saved power in an NLMS interference canceller with an approximate Newton–Raphson integer divider, and Coelho et al. [25] reduced the cost of adaptive filtering with sum-of-powers-of-two arithmetic. Yao and Huang [3] approximated the reciprocal of the input power by canonical-signed-digit encoded divisors in a 16-tap NLMS FIR filter. Huang and Yao [4] followed with a binary reciprocal approximation for an NLMS IIR filter. On the same DE2-115 board used here, their design needs 5.07 nJ per sample, compared with 8.33 nJ for a 15-bit SRT divider, and lowers the energy per sample by 39.1%, 6.5%, 14.4% and 5.6% relative to the SRT, Newton–Raphson, CSD and lookup-table methods [4]. They also remark that the older idea of rounding the divisor to a single power of two, so that the division becomes a shift, suffers from a large approximation error. That plain shift is exactly NLMS-Shift. Section 8.4 shows that its error acts only as a level-dependent step factor between 0.5 and 1.

No study was found that compares LMS, a true divider and a shift normalizer on a single core with equal tuning and statistical tests, and only a few test a complete audio system with a codec on a board. ULLAP contributes this equal-effort three-way comparison, a bit-exact chain from model to hardware, and an evaluation on real recordings.

3 System overview

Fig. 1 gives an overview of the system. Noise from the source v reaches the primary microphone through an unknown acoustic path h and mixes there with the wanted signal s, while a reference microphone close to the source picks up v directly. The two microphones feed the left and right LINE IN channels of the WM8731 codec, which digitizes both signals and sends them to the FPGA over an I^2S link. Inside the FPGA, the I^2S receiver hands one pair $(x[n], d[n])$ to the ANC core in every sample period. The core returns the error $e[n]$, that is, the cleaned signal, and the I^2S transmitter passes it back to the codec DAC and LINE OUT. An I^2C engine programs the codec once after reset. Two slide switches select bypass and freeze the adaptation, a push button resets the system, and four GPIO test points bring out the sample strobe, the output strobe, the busy flag and the frame clock so that the timing can be measured. The whole design is plain Verilog, with no processor and no vendor IP.

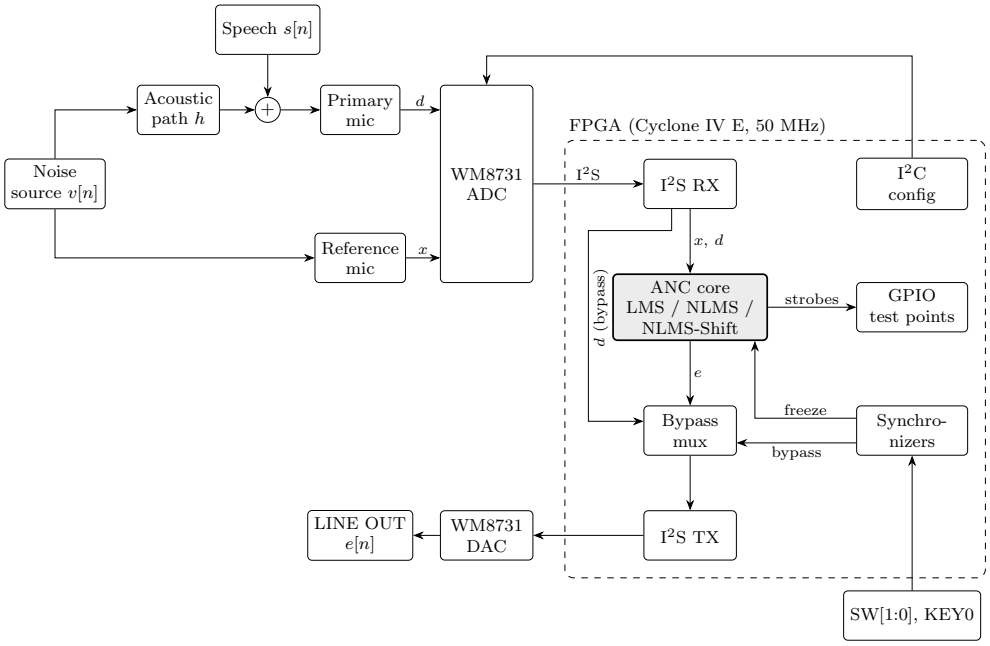


Fig. 1 Block diagram of the ULLAP system. In the experiments the acoustic part (upper left) is simulated (Section 7); everything from the codec onwards runs on the DE2-115 board.

4 Algorithms and fixed-point model

4.1 Signal model and cost function

The reference microphone hears only the noise. The primary microphone hears the speech, the noise after the acoustic path, and a little sensor noise n_s:

$$x[n] = v[n], \tag{1}$$

$$d[n] = s[n] + \sum_{j=0}^{M-1} h_j \, v[n-j] + n_s[n]. \tag{2}$$

With the last L reference samples collected in $\mathbf{x}[n] = [x[n], \ldots, x[n-L+1]]^T$ and the weights in $\mathbf{w}[n] = [w_0[n], \ldots, w_{L-1}[n]]^T$, the filter estimates the noise part of $d[n]$, and whatever remains is the system output:

$$y[n] = \mathbf{w}^T[n]\,\mathbf{x}[n] = \sum_{i=0}^{L-1} w_i[n]\, x[n-i], \tag{3}$$

$$e[n] = d[n] - y[n]. \tag{4}$$

Because s is uncorrelated with v, minimizing the mean-square error

$$J(\mathbf{w}) = E\{e^2[n]\} = E\{d^2[n]\} - 2\,\mathbf{w}^T\mathbf{p} + \mathbf{w}^T\mathbf{R}\,\mathbf{w}, \tag{5}$$

$$\mathbf{R} = E\{\mathbf{x}[n]\,\mathbf{x}^T[n]\}, \qquad \mathbf{p} = E\{d[n]\,\mathbf{x}[n]\}, \tag{6}$$

removes the noise and leaves the speech untouched. Here $\mathbf{R}$ is the autocorrelation matrix of the reference vector and $\mathbf{p}$ the cross-correlation between $d[n]$ and $\mathbf{x}[n]$. The minimum lies at the Wiener solution $\mathbf{w}_o = \mathbf{R}^{-1}\mathbf{p}$, which coincides with the acoustic path when $L \geq M$ and n_s is white.

4.2 LMS, NLMS and NLMS-Shift

LMS replaces the gradient $\nabla J = -2(\mathbf{p} - \mathbf{R}\mathbf{w})$ by its instantaneous estimate $-2e[n]\mathbf{x}[n]$, which leads to the update shared by all three algorithms:

$$\mathbf{w}[n+1] = \mathbf{w}[n] + \mu[n]\,e[n]\,\mathbf{x}[n]. \tag{7}$$

Only the step $\mu[n]$ differs. Writing the window power as $P[n] = \|\mathbf{x}[n]\|^2$, the steps of LMS, NLMS and NLMS-Shift are

$$\mu_{\mathrm{LMS}}[n] = \mu, \qquad 0 < \mu < \frac{2}{L\,\sigma_x^2}, \tag{8}$$

$$\mu_{\mathrm{NLMS}}[n] = \frac{\tilde{\mu}}{P[n] + \epsilon}, \qquad 0 < \tilde{\mu} < 2, \tag{9}$$

$$\mu_{\mathrm{Shift}}[n] = 2^{-(k[n]+K)}, \tag{10}$$

$$k[n] = \lfloor \log_2(P[n] + \epsilon) \rfloor + 1, \tag{11}$$

where σ_x^2 is the reference variance and $\epsilon > 0$ a small regularizer. The LMS bound moves with the input level, which is why a fixed μ ends up too slow for quiet inputs or unstable for loud ones. The NLMS bound stays put. The hardware keeps the power up to date through the exact recursion

$$P[n] = P[n-1] + x^2[n] - x^2[n-L], \tag{12}$$

which costs two squarings per sample instead of L. In NLMS-Shift the exponent $k[n]$ is just the bit length of $P[n] + \epsilon$. A leading-one detector finds it, and multiplying by $\mu[n]$ reduces to an arithmetic right shift of $e[n]\,x[n-i]$ by $k[n] + K$ places.

Proposition 1. *For every n, the NLMS-Shift step (10) satisfies*

$$\frac{2^{-K}}{2\,(P[n] + \epsilon)} \;\leq\; \mu_{Shift}[n] \;<\; \frac{2^{-K}}{P[n] + \epsilon}, \tag{13}$$

Put differently, the NLMS-Shift step is never more than a factor of two below the NLMS step with $\tilde{\mu} = 2^{-K}$, and it stays inside the NLMS stability bound $0 < \tilde{\mu} < 2$ whenever $K \geq 0$.

Proof. By (11), $2^{k[n]-1} \le P[n]+\epsilon < 2^{k[n]}$. Taking reciprocals and multiplying by 2^{-K} gives (13), since $\mu_{\text{Shift}}[n] = 2^{-K}\,2^{-k[n]}$. □

More precisely, the two steps are related by

$$\mu[n] = \frac{r[n]\,2^{-K}}{P[n]+\epsilon}, \qquad r[n] = \frac{P[n]+\epsilon}{2^{\lfloor \log_2(P[n]+\epsilon)\rfloor+1}} \in [0.5, 1), \tag{14}$$

so NLMS-Shift is simply NLMS with the time-varying step $2^{-K}r[n]$ (Section 8.4). When the input level is unknown over several octaves, the fractional part $u = \log_2(P+\epsilon) - \lfloor \log_2(P+\epsilon)\rfloor$ is spread evenly over $[0,1)$, and the ratio averages to

$$E\{r\} = \int_0^1 2^{u-1}\,du = \frac{1}{2\ln 2} \approx 0.72. \tag{15}$$

4.3 Fixed-point model

Signals and weights are signed $Q1.F$ numbers with $W = 12$ bits and $F = W-1$ fraction bits. With the rounding shift and the saturation

$$\text{rs}(a,m) = \left\lfloor \frac{a + 2^{m-1}}{2^m} \right\rfloor, \tag{16}$$

$$\text{sat}_B(a) = \min\!\big(\max\!\big(a, -2^{B-1}\big),\, 2^{B-1}-1\big), \tag{17}$$

the integer datapath processes the samples $x_i = x[n-i]\cdot 2^F$ and weights w_i as

$$a[n] = \text{sat}_{B_a}\Big(\sum_{i=0}^{L-1} w_i\,x_i\Big), \tag{18}$$

$$B_a = 2W + \lceil \log_2 L \rceil, \tag{19}$$

$$y[n] = \text{sat}_W\,\big(\text{rs}(a[n], F)\big), \tag{20}$$

$$e[n] = \text{sat}_W\,\big(d[n] - y[n]\big), \tag{21}$$

$$w_i \leftarrow \text{sat}_W\,\big(w_i + \Delta w_i\big), \tag{22}$$

and the weight increments of LMS, NLMS and NLMS-Shift are

$$\Delta w_i^{\text{LMS}} = \text{rs}\,\big((M_\mu\,e[n])\,x_i,\; F+10\big), \tag{23}$$

$$\Delta w_i^{\text{NLMS}} = \text{rs}\,\big(g[n]\,x_i,\; F\big), \tag{24}$$

$$g[n] = \text{sat}_{W+4}\!\left(\left\lfloor \frac{2N[n]+Q[n]}{2Q[n]} \right\rfloor\right), \tag{25}$$

$$\Delta w_i^{\text{Shift}} = \text{rs}\,\big(e[n]\,x_i,\; b(Q[n]) - F + K\big), \tag{26}$$

Here $\mu = M_\mu\,2^{-10}$, $N[n] = M_\mu\,e[n]\,2^{2F-10}$, $Q[n] = P[n] + 2^{2F-E}$ for $\epsilon = 2^{-E}$, and $b(\cdot)$ is the bit length of a positive integer. Wherever a value is narrowed, it saturates and sets a sticky flag, so no overflow can pass unnoticed. The integer model

(`py/fixedpoint.py`, `py/rtl_model.py`) uses exactly these widths and produces the golden vectors.

5 Hardware architecture

5.1 Core

Fig. 2 shows the core datapath (`rtl/anc_core.v`, vendor-neutral Verilog). The core takes one sample pair per `in_valid` strobe and does all of its arithmetic on one multiplier with a three-stage pipeline (operand registers, product register, consume stage). A finite-state machine walks through each sample: two cycles square the new and the dropped reference samples for (12) (NLMS and NLMS-Shift only), L cycles form the filter sum (18), and one cycle produces y and e (21). The step computation comes next, followed by L cycles of weight updates (22). For the gain $g[n]$, NLMS adds a sequential restoring divider; NLMS-Shift has a leading-one detector and a shift-amount register instead. A `freeze` input holds the weights while filtering continues.

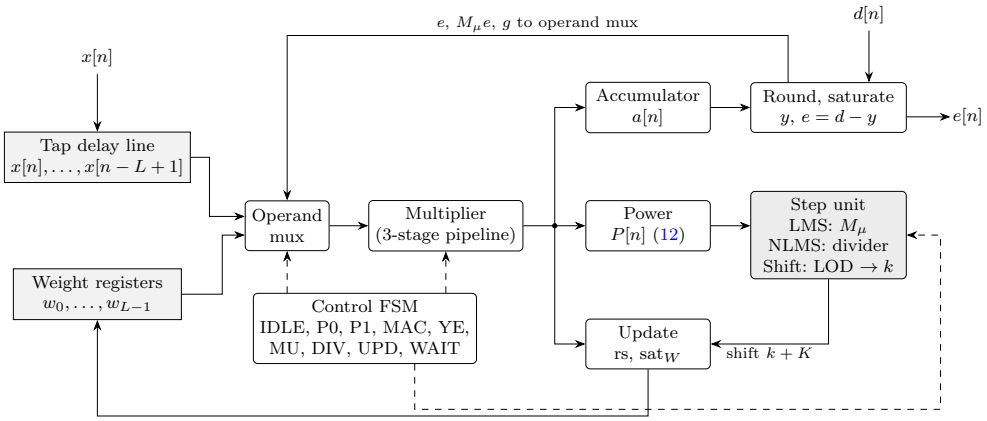


Fig. 2 Datapath of the ANC core. The power update, filter sum, step computation and weight updates all share one multiplier; the three algorithms differ only in the step unit. Dashed lines are control signals.

With this schedule, the input-to-output latency and the number of busy cycles per sample are fixed:

$$N_{\text{lat}} = \begin{cases} L+3 & \text{LMS}, \\ L+5 & \text{NLMS, NLMS-Shift}, \end{cases} \tag{27}$$

$$N_{\text{busy}} = \begin{cases} 2L+9 & \text{LMS}, \\ 2L+13+N_{\text{div}} & \text{NLMS}, \\ 2L+8 & \text{NLMS-Shift}, \end{cases} \tag{28}$$

The restoring divider takes one iteration per dividend bit, $N_{\mathrm{div}} = W + B_\mu + 2F + 4 + \lceil \log_2 L \rceil$, with B_μ the signed width of M_μ, so the NLMS cycle count grows with the step-size constant. For $L = 32$ and the tuned settings, the formulas give latencies of 35, 37 and 37 cycles and busy times of 73, 130 ($N_{\mathrm{div}} = 53$) and 72 cycles, exactly the values measured on the board (Section 8.6).

5.2 Board wrapper

Around the core, the DE2-115 wrapper holds an I^{2}C engine that configures the WM8731 at reset (11 registers, every write acknowledge-checked), an I^{2}S receiver and transmitter, two-flop synchronizers on the reset and switches, a frame-latched bypass multiplexer, an overrun flag, and four registered test points on the GPIO header. The I^{2}S blocks run on the 50 MHz system clock and treat the codec's bit and word clocks as data. The codec is the clock master. It is driven from a 12.5 MHz clock with its sample-rate register at 1536 fs, which sets the sample rate and the number of clock cycles per sample to

$$f_s = \frac{f_{\mathrm{MCLK}}}{1536} = \frac{12.5\ \mathrm{MHz}}{1536} = 8.138\ \mathrm{kHz}, \tag{29}$$

$$N_s = \frac{f_{\mathrm{clk}}}{f_s} = \frac{50\ \mathrm{MHz}}{8.138\ \mathrm{kHz}} = 6144, \tag{30}$$

where f_s is also the rate used in the simulations. The core is busy for the fraction

$$U = \frac{N_{\mathrm{busy}}}{N_s} \tag{31}$$

of every sample period; by (28) that is 1.19%, 2.12% and 1.17% for LMS, NLMS and NLMS-Shift. Again, no processor or vendor IP is involved.

6 Verification

Verification was done at three levels, and Table 1 lists what was run. At the core level, golden vectors from the integer model cover 68 configurations: word lengths W of 8 to 31 bits, filter lengths L of 2 to 32 taps, saturation corner cases and several freeze patterns. For each of them the testbench checks every $y[n]$ and $e[n]$, the final weights, the sticky flags and the latency. One level up, the top-level testbench places the board design around a behavioral model of the WM8731. This model checks each I^{2}C write and runs at whatever sample rate has been programmed, and tests T1–T10 exercise configuration, NACK handling, bypass, end-to-end cancellation against golden vectors, freeze, reset, the flags, the clocks, a 45-point skew sweep, the 8 kHz rate and the GPIO timing. A testbench that passes is only useful if it is able to fail, so faults were planted in the RTL on purpose (mutation testing); each of them was caught. Verilator, run with `-Wall`, gives no unwaived warning. The third level, the board itself, is covered in Section 8.6.

Table 1 Verification summary. All RTL suites pass in both Icarus Verilog and Questa (`results/metrics/iverilog_regression_summary.txt`, `results/local_run_da0b6f4/`).

Suite	Tests	Result
Core golden vectors (`tb_anc_core`)	68	all pass, bit-exact
Protocol corner cases	4	pass
Freeze hold/toggle variants	3	pass
Top level with codec model (T1–T10)	11	pass
Python unit tests	53	pass
Verilator lint (`-Wall`)	–	0 unwaived
On-board Signal Tap captures (T9)	25	all bit-exact
On-board freeze, DAC word, 10-min soak	4	pass

7 Experimental set-up

All synthetic runs follow the reference model $x = v$, $d = s + h * v + n_s$, sampled at 8 kHz, with 16,000 samples per run. Four scenarios are used: stationary colored noise; amplitude steps, in which the noise is scaled by $\times 3$ and later by $\times 0.25$; an acoustic path that switches abruptly at the midpoint; and a recorded chord as the noise source. Tuning was done on separate seeds (101–103). For each algorithm, the step size and regularizer were picked to give the highest mean final noise reduction over the stationary, amplitude-step and path-switch scenarios, and any setting that overflowed was discarded. This led to $\mu = 0.25$ for LMS, $\mu = 0.25$ with $\epsilon = 1$ for NLMS, and $K = 3$ with $\epsilon = 0.25$ for NLMS-Shift. Seeds 1–20, which played no part in tuning, produced every result reported below. The final noise reduction is computed over the last quarter $\mathcal{N}$ of each run,

$$\mathrm{NR} = 10 \log_{10} \frac{\sum_{n \in \mathcal{N}} \big(d[n] - s[n]\big)^2}{\sum_{n \in \mathcal{N}} \big(e[n] - s[n]\big)^2} \ \mathrm{dB}, \tag{32}$$

and the tables give its mean over the $S = 20$ seeds together with a 95% confidence interval,

$$\overline{\mathrm{NR}} \pm t_{0.975,\,S-1} \frac{\sigma_{\mathrm{NR}}}{\sqrt{S}}, \qquad t_{0.975,19} = 2.093. \tag{33}$$

When two algorithms A and B are compared, the test works on the per-seed differences $\Delta_k = \mathrm{NR}_{\mathrm{A},k} - \mathrm{NR}_{\mathrm{B},k}$, since both algorithms see the same noise realizations. A and B count as equivalent if two one-sided tests (TOST) at $p < 0.05$ both rule out a mean difference beyond $\pm\delta$, with $\delta = 0.5$ dB; that is, if both null hypotheses

$$H_{01} : \ \bar{\Delta} \le -\delta, \qquad H_{02} : \ \bar{\Delta} \ge \delta. \tag{34}$$

are rejected.

Use of generative AI. The authors used Claude (Anthropic) as an assistant during this work: for writing and debugging the analysis scripts and testbench code, for searching and summarizing the literature, and for drafting and editing the manuscript

text. [TBD: Authors: add any other AI tools you used (for example for local runs or literature searches) and adjust the list above if needed.] Every reported number was checked by the authors against the committed result files and board captures, each reference was checked against its source, and the authors take full responsibility for the content of this article.

The real-audio test uses material from the public MS-SNSD corpus[1]: four noise recordings (an air conditioner, a vacuum cleaner, a copy machine and babble) and ten utterances, all resampled to 8 kHz. For every seed, a noise segment, an utterance and a random 16-tap acoustic path are drawn and mixed at 0 dB input SNR. Each mix is processed twice, once with a clean reference and once with the speech leaking into the reference at −20 dB. The first 2 s are left for convergence, and the noise reduction, the short-time objective intelligibility measure (STOI) and the narrow-band perceptual evaluation of speech quality (PESQ, ITU-T Recommendation P.862) are computed on the rest.

8 Results

8.1 Synthetic scenarios

Table 2 Final noise reduction, dB, mean [95% CI] over seeds 1–20, $W = 12$, $L = 32$ (`results/paper_numbers.csv`).

Scenario	LMS	NLMS	NLMS-Shift
Stationary	22.39 [22.12, 22.66]	22.28 [22.01, 22.54]	22.37 [22.11, 22.63]
Amplitude steps	22.36 [21.72, 23.00]	24.16 [23.58, 24.73]	24.33 [23.88, 24.78]
Path switch	18.95 [18.81, 19.09]	18.53 [18.39, 18.67]	18.91 [18.76, 19.06]
Recorded chord	5.69 [5.66, 5.73]	5.73 [5.69, 5.76]	5.46 [5.42, 5.50]

The final noise reduction of the three algorithms is listed in Table 2. NLMS-Shift and NLMS come out level on stationary noise and on amplitude steps, and NLMS-Shift and LMS come out level on stationary noise and on the path switch. Amplitude steps are where LMS falls behind: both normalized algorithms are about 2 dB better there. With a fixed step, LMS is caught between two bad options, adapting too slowly after the drop or too noisily before it. Fig. 3 makes the trade-off visible. No LMS step size gives 22 dB on stationary noise and 24 dB on amplitude steps at the same time, while both NLMS variants manage it. The recorded chord is the one case in which NLMS-Shift loses ground, by 0.2–0.3 dB.

What counts as a tie is settled by the paired per-seed tests in Table 3. Every difference between NLMS-Shift and NLMS lies inside the ±0.5 dB equivalence margin (TOST, $p < 0.05$). Some of these differences are still statistically non-zero, the +0.38 dB on the path switch being one example, but they are small enough to count as ties. Against LMS on amplitude steps, the 1.8–2.0 dB lead of the normalized algorithms is significant ($p < 10^{-7}$).

[1] https://github.com/microsoft/MS-SNSD

Table 3 Paired differences in final NR over seeds 1–20 (`results/metrics/paired_stats.csv`).

Scenario	Pair	Mean diff. (dB)	95% CI	p (paired t)	TOST ±0.5 dB
Stationary	NLMS-Shift − NLMS	+0.10	[+0.01, +0.18]	0.028	equivalent
Stationary	NLMS-Shift − LMS	-0.02	[-0.07, +0.04]	0.58	equivalent
Stationary	NLMS − LMS	-0.11	[-0.18, -0.04]	0.0038	equivalent
Amplitude steps	NLMS-Shift − NLMS	+0.17	[-0.12, +0.47]	0.23	equivalent
Amplitude steps	NLMS-Shift − LMS	+1.97	[+1.51, +2.43]	2.8e-08	not equiv.
Amplitude steps	NLMS − LMS	+1.79	[+1.38, +2.21]	2.5e-08	not equiv.
Path switch	NLMS-Shift − NLMS	+0.38	[+0.30, +0.47]	1.9e-08	equivalent
Path switch	NLMS-Shift − LMS	-0.04	[-0.07, -0.01]	0.018	equivalent
Path switch	NLMS − LMS	-0.42	[-0.50, -0.34]	1.1e-09	equivalent
Recorded chord	NLMS-Shift − NLMS	-0.27	[-0.30, -0.24]	7.7e-14	equivalent
Recorded chord	NLMS-Shift − LMS	-0.24	[-0.26, -0.22]	2.6e-16	equivalent
Recorded chord	NLMS − LMS	+0.03	[+0.00, +0.06]	0.036	equivalent

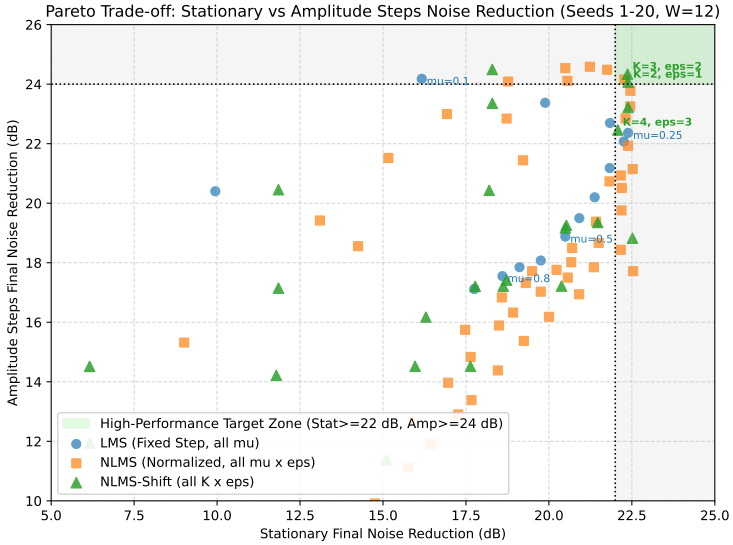


Fig. 3 Noise reduction on stationary noise against that on amplitude steps over a range of step sizes (seeds 1–20).

Figs. 4–6 follow a single run (seed 1) at the fair-tuned settings. Fig. 4 plots the running noise reduction in each scenario, Fig. 5 sets the residual error power against the input noise power for stationary noise, and Fig. 6 shows how each algorithm responds to the amplitude steps. The drop at 1.2 s is the telling moment: after it, LMS settles roughly 3 dB under the two normalized algorithms, consistent with Table 2.

8.2 Real noise and speech

The real recordings (Table 4) give a consistent picture across all four noises. Processing roughly doubles PESQ and brings STOI close to 1, and the order of the algorithms never changes: NLMS leads LMS by 0.4–0.6 dB, and NLMS-Shift trails NLMS by 0.15–0.35 dB. The noise reduction itself is lower than in Table 2. It would be easy to blame the real noise for being harder to predict, but that is not what happens. With the speech removed from the primary input, the same filters on the

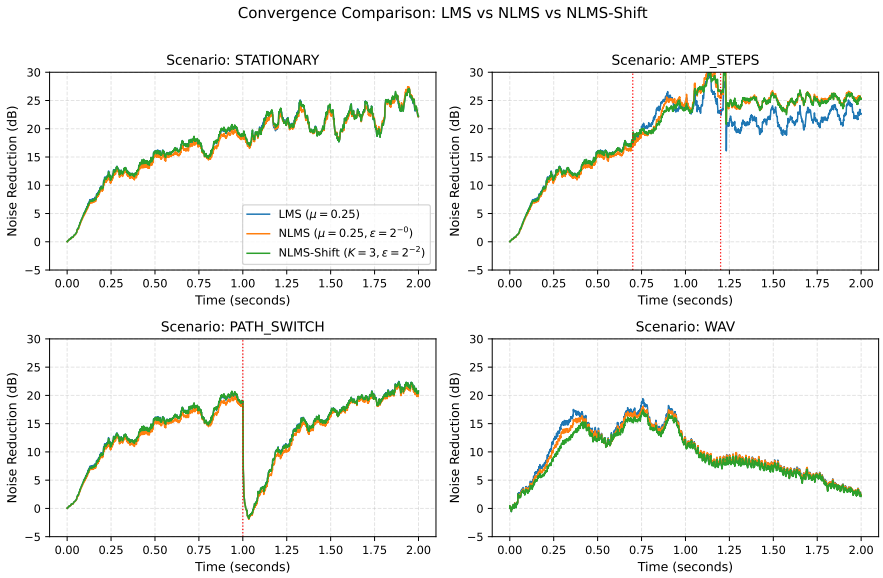


Fig. 4 Running noise reduction in the four synthetic scenarios (seed 1, $W = 12$, $L = 32$, fair-tuned settings). Dotted lines mark the amplitude steps and the path switch.

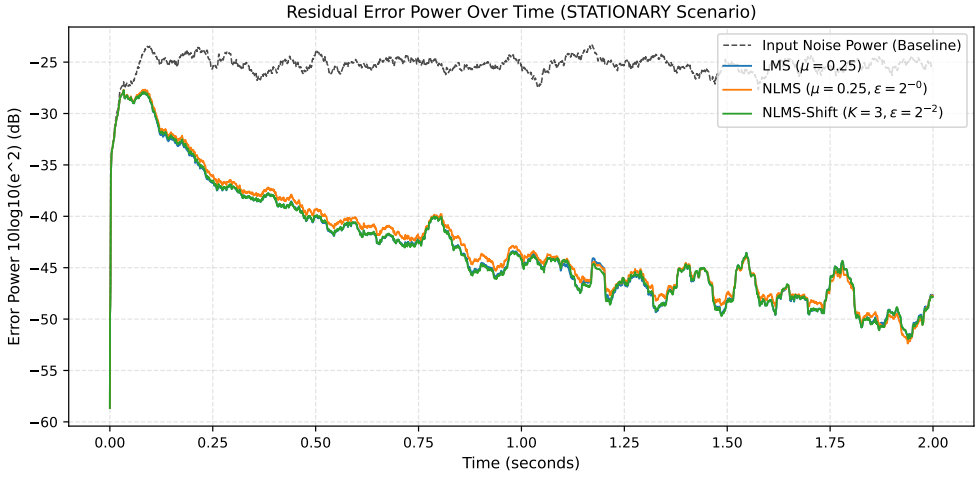


Fig. 5 Residual error power over time compared with the input noise power in the stationary scenario (seed 1).

same noise segments and paths reached 18.5–19.2 dB on the air conditioner and 22.4–31.9 dB on the other three noises, which matches or exceeds what they achieve on synthetic stationary noise (`py/exp_real_audio_nospeech.py`, `results/metrics/real_audio_nospeech.csv`). The speech is uncorrelated with the reference, yet it still reaches the error $e[n]$ that drives the weight update, and that costs 6.4–17.9 dB. LMS

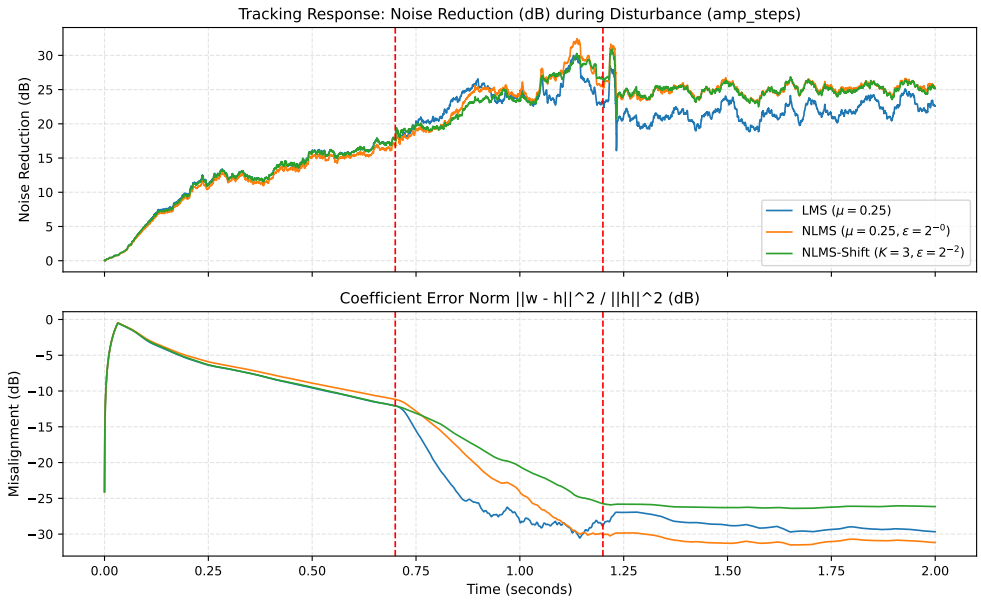


Fig. 6 Response to the amplitude-step disturbance (seed 1), shown as noise reduction and coefficient misalignment.

Table 4 Real recorded noise and speech (MS-SNSD), fixed point $W = 12$, mean ± 95% CI over 20 seeds, no speech leakage into the reference; STOI and PESQ of e against s, with the unprocessed input in brackets (`results/metrics/real_audio_summary.csv`).

Noise	Metric	LMS	NLMS	NLMS-Shift
Air conditioner	NR (dB)	11.65 ± 0.75	12.04 ± 0.82	11.89 ± 0.80
	STOI (input 0.784)	0.948	0.947	0.948
	PESQ (input 1.54)	2.34	2.30	2.33
Vacuum cleaner	NR (dB)	14.32 ± 0.32	14.90 ± 0.33	14.56 ± 0.35
	STOI (input 0.730)	0.982	0.983	0.982
	PESQ (input 1.39)	2.81	2.81	2.81
Copy machine	NR (dB)	14.02 ± 0.38	14.63 ± 0.34	14.39 ± 0.32
	STOI (input 0.694)	0.977	0.978	0.977
	PESQ (input 1.36)	2.74	2.75	2.73
Babble	NR (dB)	11.95 ± 0.42	12.43 ± 0.49	12.27 ± 0.46
	STOI (input 0.719)	0.956	0.955	0.955
	PESQ (input 1.48)	2.48	2.47	2.47

suffers most from it. Without speech, LMS is ahead of both normalized variants on all four noises; with speech, it loses the most (7.6–17.9 dB, against 6.4–14.8 dB for NLMS), so the order flips.

The standard cure is to slow down or halt adaptation during speech. In ULLAP the core's `freeze` input (Section 5.1) could be driven by a voice-activity detector for

this, but such a detector has not been tried here. Leakage of speech into the reference is the other practical concern. At −20 dB of leakage the filter starts to cancel part of the speech: noise reduction drops by 2.3–4.6 dB, PESQ by 0.25–0.6, and saturation events show up (18,500 over 19.2 million processed samples, most of them with the air conditioner). Where the reference microphone is placed therefore matters more than the choice of algorithm (`paper/tab_real_audio_xt.tex`).

8.3 Word length

Fig. 7 shows how much each algorithm loses against its own floating-point version as the word gets shorter, taking the worst scenario at the tuned settings. The loss is 4.1–5.9 dB at $W = 10$, 0.9–1.2 dB at $W = 11$ and 0.3–0.4 dB at $W = 12$. Eleven bits would already meet the 3 dB criterion; the design uses 12 to keep some margin. The tonal chord recording is the exception. On it, the fixed-point filter is up to 3.4 dB better than floating point at $W = 11$ and 1.3 dB worse at $W = 13$. With so narrow a band, quantization changes the effective adaptation, so the chord result moves around with the word length and says little about which word length to choose.

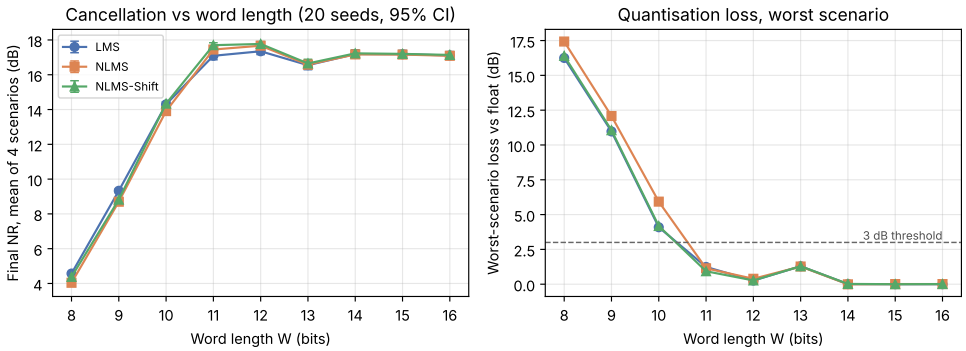


Fig. 7 Noise reduction and worst-scenario quantization loss versus word length (20 seeds).

8.4 Equivalent step size of NLMS-Shift

Equation (14) says that NLMS-Shift is NLMS with an extra factor $r[n]$ in the step. If input levels are spread evenly on a logarithmic scale, this factor averages to $E[r] = \int_0^1 2^{u-1}\,du = 1/(2\ln 2) \approx 0.72$. At any particular level, however, r depends on where $P + \epsilon$ sits inside its octave. It grows with the level and drops by half each time $P + \epsilon$ crosses a power of two. For $L = 32$ and $\epsilon = 2^{-2}$, the crossing at $P+\epsilon = 1/2$ corresponds to a reference RMS of −21.1 dBFS, only 3 dB above the level used in the simulations. The integer model was used to see what this does in practice (`py/exp_equiv_step.py`): the input level was swept over 12 dB in steps of 0.75 dB, once on the stationary scenario (seeds 1–5) and once with a two-tone reference (Fig. 8).

Broadband noise makes the window power fluctuate enough to smear out the sawtooth, and the mean ratio $\bar{r}$ drifts smoothly between 0.56 and 0.78. The two-tone

reference behaves differently. Its power P is almost constant, and $\bar{r}$ falls from 0.87 to 0.62 within a single 0.75 dB step of level. In both cases, plain NLMS run with the constant step $\bar{r}\,2^{-K}$ behaves like NLMS-Shift. Their final noise reduction agrees within 0.27 dB at all 17 broadband levels; for the tones it agrees within 0.55 dB at every level except the one that straddles the crossing (1.7 dB). Convergence times agree within 8%. NLMS with the nominal step 2^{-K}, by comparison, is off by as much as 3.5 dB and 48%.

The shift, in short, touches only the step size. Instead of a fixed NLMS step, the filter gets a step between 0.5 and 1 times the nominal one, and the input level decides where in that range it falls. With broadband input the result is a gentle change of gain, which is absorbed by choosing K (and ϵ) for the level the system expects. Tonal input is less forgiving: whenever its level crosses a power-of-two boundary of $P + \epsilon$, the step changes by up to a factor of about 0.7. A fixed-step NLMS never does this.

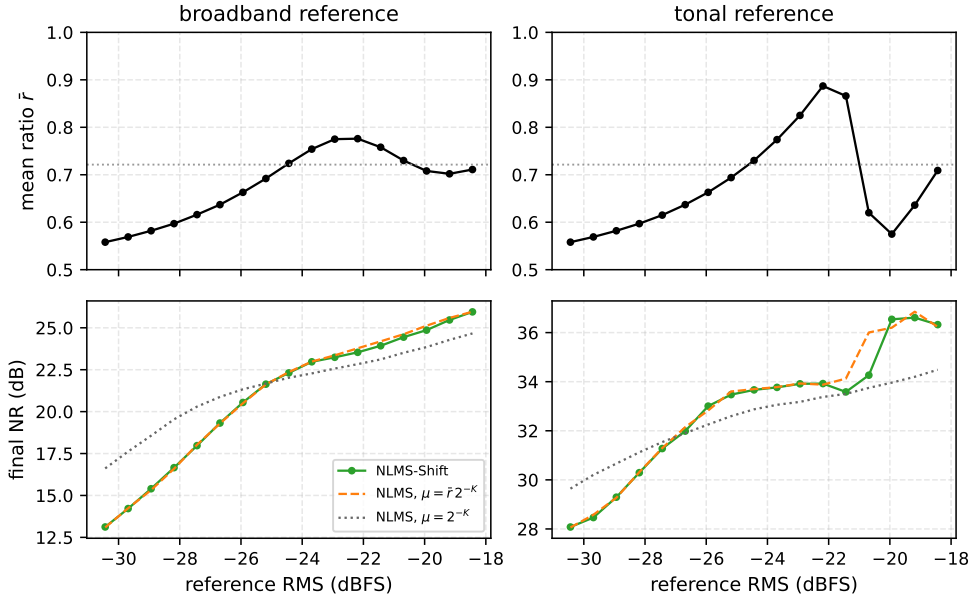


Fig. 8 Equivalent step of NLMS-Shift versus reference level (integer model, $W = 12$, $L = 32$, $K = 3$, $\epsilon = 2^{-2}$). Top: mean ratio $\bar{r}$ of (14); dotted: $1/(2 \ln 2)$. Bottom: final noise reduction of NLMS-Shift, of NLMS with step $\bar{r}\,2^{-K}$ and of NLMS with step 2^{-K}. Left: stationary scenario, seeds 1–5; right: two-tone reference (`results/metrics/equiv_step.csv`).

8.5 Hardware cost

Table 5 gives the cost of the complete board design. LMS needs about 31% less logic than either normalized variant. Between NLMS-Shift and NLMS there is almost nothing to choose in logic elements (2,610 against 2,626): what the shift saves by dropping the divider, it spends again on the leading-one detector, the shifter and a wider update

Table 5 Complete board design (core, codec interface, synchronizers, status logic) on Cyclone IV E EP4CE115F29C7, $W = 12$, $L = 32$, Quartus Prime Lite 25.1, commit da0b6f4 (`results/quartus_reports/board_da0b6f4/`, `results/metrics/power_activity.csv`).

	LMS	NLMS	NLMS-Shift
Logic elements	1,792	2,626	2,610
Registers	1,123	1,404	1,186
9-bit multipliers	4	4	4
$F_{\max}$, slow 85 °C (MHz)	70.08	59.75	58.66
Setup slack at 50 MHz, slow 85 °C (ns)	5.731	3.263	2.953
Latency (cycles)	35	37	37
Busy cycles per sample (measured on the board)	73	130	72
RTL bit toggles per sample	5,668	7,146	6,012
Core dynamic power, vectorless (mW)	20.79	24.36	23.77

path. NLMS-Shift gains in other respects. It has 15% fewer registers, needs 45% fewer busy cycles per sample and toggles 16% fewer bits per sample. Timing is not an issue for any of the three. Each meets the 50 MHz constraint with at least +2.953 ns of setup slack and +0.401 ns of hold slack, and Quartus reports no critical warnings. At 8.138 kHz even the busiest variant is active for only 2% of the 6,144-cycle sample period, so one core could handle many channels or much longer filters.

The power numbers in Table 5 need some care. They are vectorless Quartus estimates at the default toggle rate of 12.5%, and Quartus itself grades them as low confidence. Most of the 169 to 173 mW total is core static power (about 98.7 mW). The dynamic part ranks the designs LMS < NLMS-Shift < NLMS, which is also the order of the toggle counts. A fixed toggle rate, though, mainly reflects area. It has no way of knowing that the datapath sits idle for 98% of each sample period, so the figures are upper bounds and not energy per sample. A gate-level estimate with real switching activity is left for future work; it needs a separate netlist-only testbench, because the post-fit netlist flattens the hierarchy that the top-level testbench looks into. The same missing number stands in the way of a fair comparison with Huang and Yao [4], who worked on the same board. Such a comparison would use energy per sample, i.e. the dynamic power from an activity-annotated Power Analyzer run divided by the sample rate. Until that figure exists, no energy comparison with their design is claimed. Even then, area and energy would have to be compared per tap and per sample rather than as raw totals, since their design is fully parallel and runs at megasamples per second, while ULLAP shares a single multiplier and runs at 8.138 kHz.

8.6 On-board results

All board measurements were taken with Signal Tap, the logic analyzer embedded in the FPGA tool flow, so no external recorder or oscilloscope was involved. One analyzer instance stores three values once per sample: the two codec samples entering the core (x and d, from the registers `x_reg` and `d_reg`) and the core output e. A transitional storage qualifier on `out_valid` picks out one entry per sample; the trigger is the release of the reset button, and the buffer holds 32,768 samples (4.0 s). A second instance

watches the four GPIO test points at 50 MHz. For the stimuli, the noise-only versions of the four scenarios were played from a laptop into LINE IN, each preceded by a 1 kHz calibration tone. Reset was pressed while the tone was playing. Every capture therefore starts with zero weights on real codec data at about −25 dBFS, and no captured sample reaches full scale. The analysis script and the raw exports are kept in the repository (`py/analyze_board_capture.py`, `board_evidence/`).

Bit-exactness. The check itself is simple. For each of the 12 captures (three algorithms times four scenarios), the integer model of Section 4 was started from zero state at the first sample after reset, given the captured x and d, and left to compute $e[n]$. Its output matched the captured $e[n]$ sample for sample, with no mismatch anywhere in 351,723 samples (28,672 to 32,768 per capture, or 3.5 to 4.0 s). The chip, in other words, computes exactly the fixed-point arithmetic that was evaluated in simulation, and it does so on analog input carrying the codec's noise and offsets.

Timing. The GPIO capture gives an `in_valid` period of 6,144 clock cycles, i.e. 8,138.02 Hz. From `in_valid` to `out_valid` it measures 38, 36 and 38 cycles for NLMS-Shift, LMS and NLMS (0.76, 0.72 and 0.76 μs); this is the core latency plus one cycle to accept the sample. The busy times are 72, 73 and 130 cycles. Every one of these values is the same as in RTL simulation with identical parameters. Adding Signal Tap does not break timing either: those builds still meet 50 MHz, with setup slack of +2.65, +5.68 and +3.78 ns. (Table 5 refers to the builds without the logic analyzer.)

Fig. 10 puts the three algorithms side by side on the same amplitude-step stimulus, aligned to the start of the scenario. Because KEY0 was pressed by hand during the calibration tone, the reset falls at a slightly different point of the stimulus in each capture. All three reach about 15 dB within the first 1.5 s. At 2.1 s the level of d jumps by 7.4 dB, and the filters follow with only a short burst in e. After the step, the residual power (−47 to −52 dBFS) is actually lower than before it (−44 dBFS), and the noise reduction rises to 25 to 30 dB. Over the last quarter of the window, about 40 dB is removed below 1.5 kHz and 11 to 15 dB near the band edge (3.5 to 3.9 kHz).

Noise reduction on the simulation signal. In a second session the board was fed exactly the signals used in the simulations, so that its noise reduction could be compared with the model directly. For each scenario, the 2 s, seed-1 noise-only signal (the same 16,000 samples as in the simulations) was played once after the calibration tone, which lets the whole scenario fit in one capture. Correlating the captured x with the stimulus located its start (correlation 0.95 to 0.99), and $\mathrm{NR}_{\mathrm{final}}$ was taken over the same last 25 % as in simulation. All 12 captures, 28,672 samples after reset each, are bit-exact once again.

Table 6 sets board and integer model against each other on the same signal. The board ends up 2.9 to 10.6 dB below the simulation, by about the same amount for all three algorithms. The hardware is not the cause; the input level is. The codec delivered x and d at about −27 dBFS, 3 dB lower than in simulation, and with the tuned step sizes a quieter input adapts more slowly within the 2 s. When the model is fed the level the board actually received, it agrees with the board to within 0.2 dB on stationary and path-switch noise (Fig. 9). The amplitude-step and recorded-noise scenarios are harder to match because their final quarter is quiet (d near −41 and −57 dBFS), and there the board stays 1.7 to 8.1 dB below the level-matched model. At such low levels

two analog effects start to matter next to d: the noise of the playback and codec chain (-70 dBFS, measured in the silence after the stimulus) and an ADC offset that dies away during the first second after reset. A simple check on the amplitude-step window shows how much this limits the result. Even the best 32-tap least-squares filter fitted to the board samples of that window reaches only 24 dB, against 28 dB on the ideal signal. Across all scenarios on the board, the three algorithms stay within 0.7 dB of one another.

Table 6 On-board $\mathrm{NR}_{\mathrm{final}}$ (dB) on the 2 s seed-1 stimulus against the integer model on the same signal, at the simulation level and scaled to the level the board received (`results/board/session2/`). Every board capture is bit-exact.

Algorithm	Scenario	Board	Model, sim. level	Model, board level
LMS	stationary	15.88	22.77	15.76
	amplitude steps	17.13	27.71	25.23
	path switch	14.67	19.84	14.47
	recorded noise	5.99	9.11	7.78
NLMS	stationary	15.65	21.60	15.53
	amplitude steps	16.76	27.19	23.64
	path switch	14.03	19.06	14.04
	recorded noise	5.80	8.72	7.45
NLMS-Shift	stationary	15.80	22.68	15.78
	amplitude steps	16.57	25.79	22.10
	path switch	14.40	19.86	14.44
	recorded noise	5.46	8.63	7.32

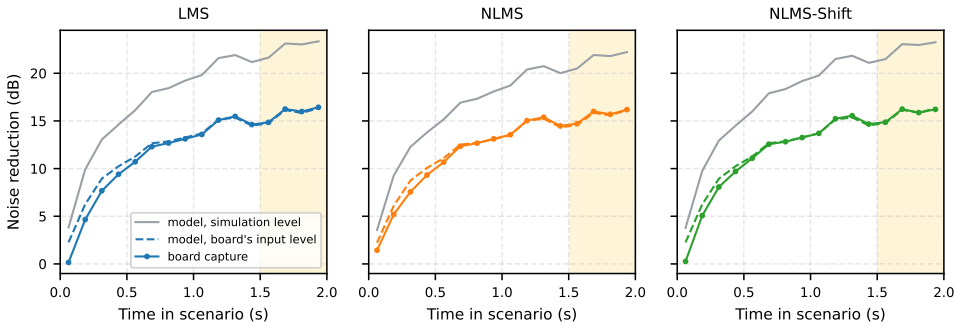


Fig. 9 Running noise reduction (0.125 s windows) on the 2 s stationary stimulus: board capture, integer model at the simulation level, and integer model with the stimulus scaled to the level the board received. Shaded: the $\mathrm{NR}_{\mathrm{final}}$ window.

Freeze, DAC word and endurance. To test the freeze path, SW[1] was flipped by hand 1.44 s into a stationary-noise run with NLMS-Shift. The captured output equals

the model with adaptation stopped from that sample onwards (28,672 samples, no mismatch). From the switch to the end of the run, the noise reduction holds at 15.2 dB; just before the switch it was 15.4 dB, and with adaptation left running it would have been 15.9 dB. A third Signal Tap instance looked at the word sent to the DAC. In ANC mode this word is $\{e, 0000\}$, and in bypass (SW[0] = 1) its upper 12 bits are d, in all 1,792 samples of each capture. The analog LINE OUT signal could not be recorded with the equipment available, so the check ends at the DAC interface. Finally, the board was left running for 10 minutes on looped noise. Afterwards the GPIO timing was unchanged (6,144 cycles per sample, 38 cycles from `in_valid` to `out_valid`, 72 busy cycles), and none of the saturation, NACK or overrun flags had been set. With all three analyzer instances included, the build still met timing (slack +2.93 ns).

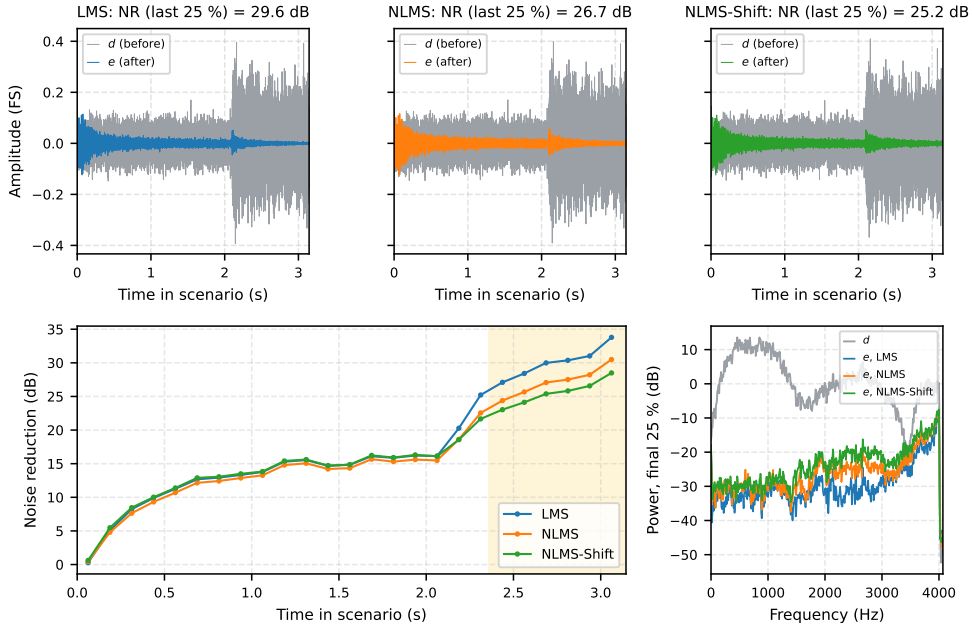


Fig. 10 On-board Signal Tap captures of LMS, NLMS and NLMS-Shift on the same amplitude-step noise, from the start of the scenario: primary input d and output e (top), running noise reduction in 0.125 s windows (bottom left; shaded: last 25 % of the common window) and power spectra over that window (bottom right). Every output sample of all three captures equals the integer model's.

9 Discussion and limitations

What the comparison supports is a fairly narrow statement. A divider-free normalization keeps the level independence of NLMS while costing roughly the cycles of LMS. That makes it the better option when the noise level varies and throughput or energy per sample is a concern; for a steady level, LMS is still the smallest design. Nothing here shows that NLMS-Shift cancels better than NLMS, and Section 8.4 explains

why. The shift changes only the step size, by a level-dependent factor between 0.5 and 1, so tuning K and ϵ for the expected input level matters more than the shift itself. Tonal noise close to a power-of-two boundary of $P + \epsilon$ also brings a step jump that NLMS does not have. With real recordings, the speech at the primary input limits all three algorithms much more than the choice among them does (6.4–17.9 dB, against at most 0.6 dB between algorithms).

The study has limits that should be kept in mind. The acoustic paths are simulated, also in the tests with real recordings. Acoustic feedback (closed-loop howling) is not covered, and reference-based ANC would be the wrong structure for it in any case. Power is only a switching-activity proxy until a gate-level analysis has been done. On the board, the output was shown to equal the model up to the DAC word, but the analog LINE OUT signal was not recorded. Lastly, the absolute noise reduction depends on the input level: with the stimulus 3 dB below the simulation level, the board reached 14 to 17 dB on the synthetic scenarios instead of 19 to 28 dB, and for stationary and path-switch noise the model reproduces this at that level. A practical system should therefore set the codec input gain so that the reference fills most of the 12-bit range, or retune the step size for the level it expects.

10 Conclusion

ULLAP runs LMS, NLMS and NLMS-Shift on one serial 12-bit, 32-tap core on a low-cost Cyclone IV board, and this paper compared the three with equal tuning effort. NLMS-Shift matches NLMS within the 95% confidence interval on stationary and amplitude-step noise, while needing 72 busy cycles per sample instead of 130. On real recorded noise all three remove 11.7 to 14.9 dB; once the speech is taken out of the primary input they remove 18.5 to 31.9 dB, so it is the speech, not the noise, that limits them. NLMS-Shift was also shown to be equivalent to NLMS with a level-dependent step between 0.5 and 1 times the nominal one. On the board, 25 Signal Tap captures of real codec data (724,459 samples) are bit-exact with the integer model, as are the freeze behavior and the DAC word. For stationary and path-switch noise, the measured noise reduction matches the model at the input level the board received, to within 0.2 dB.

Declarations

Funding. [TBD: Authors: funding sources, or "No funding was received for this work."]
Competing interests. [TBD: Authors: competing interests, or "The authors declare no competing interests."]
Author contributions. [TBD: Authors: who did what (free text or CRediT).]
Data and code availability. All RTL, testbenches, scripts and result files are available in the project repository [TBD: public URL and Zenodo DOI of the tagged release]. The real-audio study downloads its MS-SNSD files from a pinned commit and verifies them by SHA-256.
Use of generative AI. Described in the experimental set-up (Section 7).

References

[1] Lu, L., Yin, K.-L., Lamare, R.C., Zheng, Z., Yu, Y., Yang, X., Chen, B.: A survey on active noise control in the past decade–Part I: Linear systems. Signal Processing **183**, 108039 (2021) https://doi.org/10.1016/j.sigpro.2021.108039

[2] Guidotti, V., Paim, G., Rocha, L.M.G., Costa, E., Almeida, S., Bampi, S.: Power-efficient approximate Newton-Raphson integer divider applied to NLMS adaptive filter for high-quality interference cancelling. Circuits, Systems, and Signal Processing **39**, 5729–5757 (2020)

[3] Yao, C.-Y., Huang, Y.-Z.: The design of the NLMS adaptive filters using the fast-division approximation with CSD encoded divisors. IEEE Transactions on Circuits and Systems II: Express Briefs **71**(4), 2459–2463 (2024)

[4] Huang, Y.-Z., Yao, C.-Y.: Enhancing the power efficiency of the NLMS adaptive IIR filter with fast binary reciprocal approximation. Circuits, Systems, and Signal Processing **45**, 7561–7584 (2026) https://doi.org/10.1007/s00034-026-03536-z

[5] Münzel, T., Kuntic, M., Molitor, M., Sørensen, M., Daiber, A.: Transportation noise pollution as a cardiovascular risk factor: from epidemiological evidence to mechanistic insights. EXCLI Journal **24**, 1713–1739 (2025) https://doi.org/10.17179/excli2025-9050

[6] Hahad, O., Kuntic, M., Al-Kindi, S., Kuntic, I., Gilan, D., Petrowski, K., Daiber, A., Münzel, T.: Noise and mental health: evidence, mechanisms, and consequences. Journal of Exposure Science and Environmental Epidemiology **35**(1), 16–23 (2025) https://doi.org/10.1038/s41370-024-00642-5

[7] Chadha, S., Kamenov, K., Cieza, A.: The world report on hearing, 2021. Bulletin of the World Health Organization **99**(4), 242–242 (2021) https://doi.org/10.2471/BLT.21.285643

[8] Yang, B., Yin, J., Ye, Z., Yang, S., Wang, L.: Development and testing of an active noise control system for urban road traffic noise. Applied Sciences **14**(1), 175 (2024) https://doi.org/10.3390/app14010175

[9] Jiang, T., Liu, J., Peng, C., Wang, S.: Laboratory test of a vehicle active noise-control system based on an adaptive step size algorithm. Applied Sciences **13**(1), 225 (2023) https://doi.org/10.3390/app13010225

[10] Zhu, L., Zhong, X., Wang, H., Qiu, X., Wu, S., Mao, D.: An objective and subjective study on global active control of engine noise in a car. Journal of Vibration and Control **30**(9–10), 2311–2321 (2024) https://doi.org/10.1177/10775463231174730

[11] Zhu, W., Gu, Z., Chen, X., Xie, P., Luo, L., Bai, Z.: A new hybrid adaptive

self-loading filter and GRU-net for active noise control in a right-angle bending pipe of an air conditioner. Sensors **25**(20), 6293 (2025) https://doi.org/10.3390/s25206293

[12] Yin, Y., Chen, F.: Acoustic feedback cancellation algorithm for hearing aids based on a weighted error adaptive filter. Electronics **12**(7), 1528 (2023) https://doi.org/10.3390/electronics12071528

[13] Husstedt, H., Hilgerdenaar, W., Frenz, M., Denk, F., Tchorz, J.: Evaluation of impulse noise reduction in hearing aids with technical measurements and ratings of discomfort. Acta Acustica **7**, 47 (2023) https://doi.org/10.1051/aacus/2023042

[14] Tetteh, P.D., Gyening, R.-M.O.M., Arthur, J., Owusu, J.A., Kponyo, B.A., Ahene, E., Kponyo, J.J.: Speech enhancement techniques for hearing aids: a systematic review. Universal Access in the Information Society **25**(3), 75 (2026) https://doi.org/10.1007/s10209-026-01342-7

[15] Lu, L., Yin, K.-L., Lamare, R.C., Zheng, Z., Yu, Y., Yang, X., Chen, B.: A survey on active noise control in the past decade–Part II: Nonlinear systems. Signal Processing **181**, 107929 (2021) https://doi.org/10.1016/j.sigpro.2020.107929

[16] Félix, F.B., Castro Magalhães, M., Souza Papini, G.: Improved active noise control algorithm based on the convex combination method. Journal of the Brazilian Society of Mechanical Sciences and Engineering **43**, 163 (2021) https://doi.org/10.1007/s40430-021-02866-0

[17] Li, F., Ding, H., Wu, Z., Qian, F., Liu, T., Iqbal, A.: An improved stability compensation for feedforward active noise control systems with acoustic feedback. Journal of Mechanical Science and Technology **38**, 507–518 (2024) https://doi.org/10.1007/s12206-024-0101-5

[18] Thannoon, H.H., Hashim, I.A.: Hardware implementation of a high-speed adaptive filter using a combination of systolic and convex architectures. Circuits, Systems, and Signal Processing **43**, 1773–1791 (2024) https://doi.org/10.1007/s00034-023-02539-4

[19] Jagadeeswari, M., Rishabavarthani, P., Kumarganesh, S., Kannadhasan, S.: Enhanced adaptive filtering in audio systems: normalized least mean square with cut-set retiming. Analog Integrated Circuits and Signal Processing **126**(1), 12 (2026) https://doi.org/10.1007/s10470-025-02537-3

[20] Karakaya, E., Aytar, O.: FPGA implementation of NLMS and VSLMS algorithms for underwater acoustics applications. IEEE Access **14**, 53860–53872 (2026) https://doi.org/10.1109/ACCESS.2026.3681282

[21] Park, S., Park, D.: Low-power FPGA realization of lightweight active noise cancellation with CNN noise classification. Electronics **12**(11), 2511 (2023) https:

//doi.org/10.3390/electronics12112511

[22] Abdi, F., Cichalewski, W., Jałmużna, W., Butkowski, Ł., Branlard, J., Bellandi, A., Jabłoński, G.: FPGA-based implementation of an adaptive noise controller for continuous wave superconducting cavity. Electronics **13**(1), 155 (2024) https://doi.org/10.3390/electronics13010155

[23] Nagabushanam, M., Chakrasali, S., Gangadharaiah, S.L., Patel, S.H., Ramaiah, G., Basak, R.: An optimized VLSI implementation of the least mean square (LMS) adaptive filter architecture on the basis of distributed arithmetic approach. Journal of The Institution of Engineers (India): Series B **106**(3), 861–870 (2025) https://doi.org/10.1007/s40031-024-01028-9

[24] Rosa, M.M.A., Costa, P.Ü., Costa, E.A.C., Almeida, S.J.M., Paim, G., Bampi, S.: Design of a low power and robust VLSI power line interference canceler with optimized arithmetic operators. Analog Integrated Circuits and Signal Processing **112**(2), 247–261 (2022) https://doi.org/10.1007/s10470-022-02050-x

[25] Coelho, L.F.d.S., Lovisolo, L., Tcheou, M.P.: Adaptive filtering with reduced computational complexity using SOPOT arithmetic. IEEE Transactions on Circuits and Systems I: Regular Papers **69**(2), 746–756 (2022)